

N-CAPAS BY MODULE Y N-CAPAS ALL PROJECT

Programa: Ingeniería De Sistemas

Asignatura: Arquitectura De Software

Profesor: Jesús Ariel Gonzales

Estudiante

María Sofia Aljure Herrera

Corporación Universitaria Del Huila Corhuila

Neiva Huila

Octubre 2025

Arquitectura N-Capas arquitectura en N-Capas es un patrón clásico de diseño de software que organiza el sistema en capas jerárquicas, donde cada capa tiene una responsabilidad específica y conoce únicamente la capa inmediatamente inferior.

Esta arquitectura tiene un flujo típico que es:

1. **Presentación (Controller / API Layer):** esta parte Interactúa con el usuario o cliente externo, recibe las peticiones, valida datos y los redirige a la lógica de negocio.
2. **Aplicación / Servicio (Service Layer):** acá esta la lógica de negocio, define las interfaces (IService) y clases que implementan esa lógica (ServiceImpl).
3. **Dominio / Modelo (Domain Layer / Entity Layer):** en esta parte se definen las entidades o modelos que representan objetos del negocio (ej. User, Product, Order) y se encargan de reflejar reglas básicas del negocio.
4. **Persistencia / Integración (Repository Layer / Data Access Layer):** la persistencia o integración encapsula la lógica de acceso a datos, normalmente implementada con repositorios que extienden de JPA/Hibernate (JpaRepository).

Formas de organizar el código con N-Capas

Cuando aplicamos N-Capas en un monolito o en un modular monolith, existen dos formas principales de organizar los paquetes del proyecto:

N-Capas all project

En este enfoque, todos los controllers, services, repositories y model están agrupados en carpetas globales. Las capas están centralizadas y todos los features comparten esas capas.

Ejemplo de estructura:

- Proyecto único con paquetes por capa:
 - com.app.presentation
 - com.app.application
 - com.app.domain
 - com.app.persistence

Ventajas

- **Simplicidad inicial:** es una estructura conocida, tutoriales y las herramientas la favorecen.

- **Menos artefactos en el build:** el que haya un solo jar/war facilita despliegue inicial.
- **Reutilización centralizada:** las clases utilitarias y modelos se ubican en un solo sitio.

Desventajas

- **Crecimiento en espiral:** a medida que el sistema crece, los paquetes por capa se vuelven enormes y difíciles de navegar.
- **Acoplamiento por tipo:** la tendencia es que módulos funcionales queden dispersos por múltiples paquetes (controlador, servicio, repositorio), dificultando cambios locales.
- **Dificultad para dividir en microservicios:** separar funcionalidades en servicios físicos requiere muchas refactorizaciones y extraer código con riesgo.

Tipos de proyectos indicados

- Aplicaciones pequeñas o medianas donde la cantidad de funcionalidades es reducida.
- Proyectos de uso interno en una empresa.
- Prototipos o MVPs donde lo más importante es salir rápido a producción.
- Aplicaciones académicas o de práctica (proyectos universitarios, laboratorios).

N-Capas by module

En este enfoque, la aplicación se divide en módulos funcionales (features) y cada módulo tiene sus propias capas internas (controller, service, repository, model).

Por ejemplo: module-product tiene product.controller, product.service, product.repository, product.model.

Ejemplo de estructura:

- Módulos (ejemplos): product-module, order-module, user-module
- Dentro de product-module:
 - product.controller
 - product.service
 - product.domain / product.model
 - product.repository
 - product.integration

Como cada módulo tiene sus propias capas internas es responsable por su propio ciclo de vida lógico y puede ser construido, testeado y algunas veces desplegado independientemente si la estructura lo permite.

Ventajas

- **Cohesión alta:** Alta cohesión por módulo.
- **Facilidad para escalar el equipo:** equipos pueden trabajar por módulos con conflictos mínimos.
- **Migración a microservicios más simple:** la delimitación del módulo facilita convertirlo en servicio independiente cuando sea necesario.
- **Mejor entendimiento en contexto del dominio:** navegabilidad intuitiva por feature.

Desventajas

- **Duplicación potencial:** sin una capa común, utilidades o DTOs pueden repetirse si no se gestionan como librerías compartidas.
- **Mayor número de artefactos:** si cada módulo produce su propio jar, el sistema de build y despliegue se vuelve más complejo.
- **Comunicación entre módulos:** si hay dependencias fuertes, se pueden formar acoplamientos cruzados.

Tipos de proyectos indicados

- Aplicaciones medianas o grandes con múltiples funcionalidades y dominios de negocio.
- Proyectos donde trabajan varios equipos en paralelo, cada uno encargado de un módulo.
- Sistemas con una visión de crecimiento a futuro o donde se prevé migrar a una arquitectura de microservicios.
- Aplicaciones que requieren alta mantenibilidad y baja dependencia entre componentes.

Consumo de recursos a nivel de infraestructura

Hay que tener en cuenta que el consumo de recursos a nivel de infraestructura en N-Capas all project y N-Capas by module no es exactamente el mismo, porque la forma en que se organizan afecta la ejecución, despliegue y escalabilidad.

N-Capas all Project

La infraestructura de todo el sistema de all Project corre en un solo artefacto ya sea .jar o .war, también la aplicación es monolítica, lo que significa que se despliega como una unidad en un solo servidor o contenedor y usa una única base de datos compartida para todos los módulos y features.

Así que el consumo de recursos se puede ver de esta manera:

- **Memoria RAM y CPU:** es moderado, ya que solo existe una instancia centralizada.
- **Infraestructura:** solo basta un servidor físico, una máquina virtual o un contenedor Docker.
- **Escalabilidad:** es vertical, si la aplicación crece, se necesita más hardware (RAM, CPU) en la misma máquina.
- **Problema:** aunque solo crezca una parte del sistema (ej: módulo de pagos), se escala todo el monolito, lo cual es costoso e ineficiente.

Como decía anteriormente en los tipos de proyectos indicados es ideal para proyectos pequeños/medianos porque requiere menos infraestructura inicial.

N-Capas by Module

La infraestructura de la aplicación sigue siendo monolítica, pero internamente está organizada por módulos y cada módulo tiene su propia lógica de negocio y capas, lo que facilita desacoplarlos más adelante. Si se da el salto a microservicios, cada módulo podría convertirse en un servicio independiente con su propia base de datos o instancia.

Así que el consumo de recursos se puede ver de esta manera

- **Memoria RAM y CPU:** al inicio, es similar al all project, porque todo se despliega en un único artefacto.
- **Infraestructura inicial:** es igual de sencilla, un servidor o contenedor único.
- **Escalabilidad:** es mucho mejor preparada, puedes escalar por módulo (si se migra a microservicios) y el consumo de recursos puede distribuirse entre varios nodos o contenedores en el futuro.
- **Problema:** requiere más disciplina en el diseño y, al crecer, puede implicar mayor consumo global si cada módulo se despliega separado (ya que cada microservicio necesita memoria, CPU, base de datos o al menos un esquema independiente).

Como decía anteriormente en los tipos de proyectos indicados es ideal para proyectos medianos/grandes porque optimiza la escalabilidad futura, aunque a nivel de infraestructura el costo puede subir si se migra a microservicios.

Construcción del esqueleto del proyecto en spring boot

En este caso usare el ejercicio Flight Management System – Refined Data Mode que usamos el corte #1

<https://drive.google.com/file/d/1cKHRroX4PdvvYQDcFEWny3TxmKOAqoOs/view?usp=sharing>

Una de las preguntas claves es si mi proyecto tiene una visión de crecimiento rápido y si en un corto plazo pretende acceder hasta una arquitectura de micro-service y si es así cuál de los dos escenarios n-capas by module y n-capas all project el ideal.

En este caso si es así, el sistema de gestión de vuelos involucra muchos módulos, entidades y muchas áreas funcionales (vuelos, aviones, pasajeros, boletos, aeropuertos, etc.). Además, por naturaleza, este tipo de sistema debe crecer rápido ya que cada día se generan cientos de vuelos, también cada vuelo tiene muchos pasajeros, boletos y equipajes etc.

Como este sistema maneja varias áreas funcionales, Elegí usar la arquitectura By Module porque me ayuda a tener el sistema de gestión de vuelos mucho más ordenado. Cada módulo cumple con una función clara y no se mezcla con los demás, lo que significa que hay cohesión. Al mismo tiempo, los módulos solo se comunican cuando es necesario y sin depender demasiado unos de otros, lo que asegura un bajo acoplamiento. Esto hace que el sistema sea más fácil de entender, mantener y que, en el futuro, pueda desplegarse, optimizarse y escalarse de forma separada como microservicio.

Esta decisión la tomé apoyándome en lo expuesto en el artículo “*On Software Modular Architecture: Concepts, Metrics and Trends*” (International Journal of Computer and Organization Trends, 2022), donde se destacan precisamente estas ventajas del diseño modular.

Módulos y Entidades

1. Geolocation

Maneja la jerarquía geográfica mundial para ubicar los aeropuertos.

- **Continent:** continentes.
- **Country:** países, con referencia al continente.
- **State:** estados o provincias dentro de un país.
- **City:** ciudades, vinculadas a un estado.

2. Infrastructure

Modela la infraestructura aeroportuaria.

- **Airport:** aeropuertos, nombre y dirección.
- **Terminal:** terminales de un aeropuerto.
- **BoardingGate:** puertas de embarque dentro de una terminal.

3. Parameterization

Define catálogos y parámetros que estandarizan la información.

- **DocumentType:** tipos de documento.
- **FlightType:** tipos de vuelo.
- **AircraftType:** tipos de aeronaves.
- **CrewRole:** roles de la tripulación.
- **TicketClass:** clases de tiquetes.

4. Security

Administra la seguridad, roles de usuario y acceso al sistema.

- **Role:** roles del sistema.
- **Module:** módulos del sistema a los que se puede acceder.
- **Form:** formularios o pantallas de la aplicación.
- **FormModule:** relación entre formularios y módulos.
- **RoleForm:** asignación de permisos.
- **Person:** datos personales.
- **UserAccount:** usuarios del sistema.

5. Human Resources

Gestión del personal de la aerolínea.

- **Employee:** empleados (salario, fecha de contratación, rol de tripulación, persona asociada).

6. Management

Administración de aeronaves.

- **Aircraft:** datos de los aviones.

7. Flight Operations

Núcleo de la operación aérea, gestiona vuelos, tripulación, boletos y equipaje.

- **Flight:** datos del vuelo.
- **CrewAssignment:** asignación de empleados/roles a un vuelo específico.
- **Ticket:** boletos.
- **Baggage:** equipaje asociado a un ticket.

8. Passengers & Services

Información de los clientes de la aerolínea.

- **Passenger:** pasajeros frecuentes o únicos.

9. Notifications

Manejo de alertas y notificaciones dentro del sistema.

- **MessageTemplate:** plantillas de mensajes.
- **Notification:** mensajes enviados.

Referencias

<https://ijcotjournal.org/archive/ijcot-v12i1p302>

- [1] (2021). Britannica T, Editors of Encyclopaedia, Software. Encyclopedia Britannica. [Online]. Available: <https://www.britannica.com/technology/software>
- [2] Garlan D, & Shaw M, An Introduction to Software Architecture. In Advances in Software Engineering and Knowledge Engineering. (1993) 1-39.
- [3] (2010). Chapter 3: Architectural Patterns and Styles. [Online]. Available: [https://docs.microsoft.com/en-us/previous-versions/msp-n-p/ee658117\(v=pandp.10\)?redirectedfrom=MSDN](https://docs.microsoft.com/en-us/previous-versions/msp-n-p/ee658117(v=pandp.10)?redirectedfrom=MSDN)
- [4] (2014). Rengiaiah P, On Modular Architectures. Medium. [Online]. Available: <https://medium.com/on-software-architecture/on-modular-architectures-53ec61f88ff4>.
- [5] (2021). Accelerator (software) - Wikipedia. [Online]. Available: [https://en.wikipedia.org/wiki/Accelerator_\(software\)](https://en.wikipedia.org/wiki/Accelerator_(software))
- [6] Mehta M. R, Lee S, & Shah J. R, Service-Oriented Architecture: Concepts and Implementation. In Proceedings of the Information Systems Education Conference (ISECON). 23(2335) (2006) 1.
- [7] Tutusani T, Effective Software Development for Enterprise BeyondDDD, Software Architecture, and XP 1st edition: Leanpub. (2020).
- [8] Narduzzo A, & Rossi A, Modular Design and the Development of Complex Artefacts: Lessons from Free/Open Source Software, Quaderno, DISA. 78 (2003).
- [9] Knoernschild K, Java Application Architecture: Modularity Patterns with Examples Using OSGi. Prentice-Hall Press. (2012).
- [10] (2021). Tutorialspoint, Software Architecture & Design Tutorial - Tutorialspoint. [Online] Available: https://www.tutorialspoint.com/software_architecture_design/index.htm
- [11] Giakoumidis N, Bak J. U, Gómez J. V, Llenga A, & Mavridis N, Pilot-Scale Development of a UAV-UGV Hybrid with Air-Based UGV Path Planning. In 2012 10th International Conference on Frontiers of Information Technology IEEE. (2012) 204-208.
- [12] Schilling M. A, Toward a General Modular Systems Theory and its Application to Interfirm Product Modularity, Academy of Management Review. 25(2) (2000) 312-334.
- [13] (2020). Mustafic A, Modular Application Architecture. [Online]. Available: Goetas.com <https://www.goetas.com/modular-application-architecture-intro>
- [14] (2021). Package Development - Laravel - The PHP Framework for Web Artisans. [Online]. Available: <https://laravel.com/docs/8.x/packages>

- [15] Soothram S, Efficient Techniques for Partitioning Software Development Tasks. (2010).
- [16] (2021). Kibabii, Kibabii University. [Online]. Available: <https://kibu.ac.ke/kibu-automation-dream-accomplished-in-2019-2020-academic-year>.
- [17] (2021). Moodle, Moodle Plugins Directory. [Online]. Available: Moodle.org <https://moodle.org/plugins>
- [18] (2021). OpenMRS I, OpenMRS. [Online]. Available: Openmrs.org <https://openmrs.org>
- [19] (2020). Brinkman S, & Delamore D, The 5 Essential Elements of Modular Software Design, Medium. [Online]. Available: <https://shanebdavis.medium.com/the-5-essential-elements-of-modular-software-design-6b333918e543>.
- [20] Nesnas I. A. D, Simmons R, Gaines D, Kunz C, Diaz-Calderon A, Estlin T, Madison R, Guineau J, McHenry M, Shu I.-H, & Apfelbaum D, CLARAty: Challenges and Steps toward Reusable Robotic Software, International Journal of Advanced Robotic Systems. (2006). <https://doi.org/10.5772/5766>
- [21] Sant'Anna C, Figueiredo E, Garcia A, & Lucena C. J, On the Modularity of Software Architectures: A Concern-Driven Measurement Framework. In European Conference on Software Architecture, Springer, Berlin, Heidelberg. (2007) 207-224.
- [22] Santos D, de Resende A. M. P, Lima E. C, & Freire A. P, Software Instability Analysis Based on Afferent and Efferent Coupling Measures. J. Softw., 12(1) (2017).
- [23] Budd T, Introduction to Object-Oriented Programming, Pearson Education India. (2008).
- [24] Page-Jones M, The Practical Guide to Structured Systems Design. Yourdon Press. (1988).
- [25] (2021). Coupling (computer programming) - Wikipedia. [Online]. Available: [https://en.wikipedia.org/wiki/Coupling_\(computer_programming\)](https://en.wikipedia.org/wiki/Coupling_(computer_programming))
- [26] Northrop L, Trends and New Directions in Software Architecture, Carnegie-Mellon Univ Pittsburgh PA Software Engineering Inst. (2014)
- [27] Woods E, Software Architecture in a Changing World, IEEE Software. 33(6) (2016) 94-97.
- [28] Garlan D, Software Architecture: A Travelogue, In Future of Software Engineering Proceedings. (2014) 29-39.